

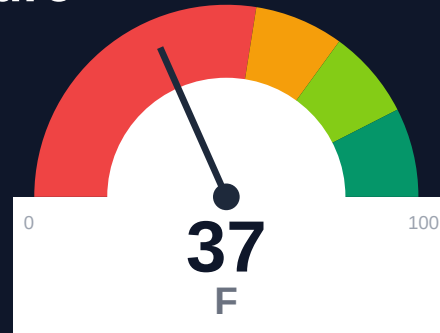
Fireblocks

Documentation Infrastructure Audit Report

Prepared by VeriOps

2026-06-24 09:50 UTC

CONFIDENTIAL | FOR INTERNAL USE ONLY



Risk: Critical

Key Metrics

Pages scanned: 724

Report type: Premium Executive Audit

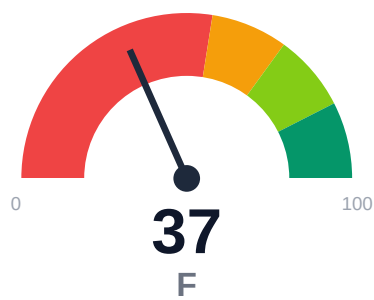
Methodology: 5-pillar automated analysis + expert synthesis

Headline Findings

- Broken internal links: 21
- Link verification inconclusive for 3 URLs (auth/rate-limit/server block); these are excluded
- SEO/GEO issue rate is high: 27.62%
- API usage-doc coverage is low: 0.61% (622 API pages detected)

This report is generated automatically by the VeriOps platform. Estimates are directional.

Executive Summary



Fireblocks developer documentation spans 724 crawled pages with 21 confirmed broken internal links and 3 unverified URLs. API reference coverage is critically low at 0.61% despite 622 API pages detected, indicating severe usage documentation gaps. Example code reliability scored 0% across all 724 pages, flagged as a likely detection limitation for JavaScript-rendered blocks. Retrieval readiness is weak (58.62/100) driven by 85.41% stale risk and only 7.73% of pages displaying last-updated metadata. SEO/geo issues affect 27.62% of pages, and only 73.65% of 983 key claims include supporting evidence.

External posture: SEO/GEO issue rate **27.6%**, public API coverage **0.6%**, broken links **21**.

37 Audit Score	F Grade	Critical Risk Band
724 Pages Scanned	21 Broken Links	27.6% SEO Issue Rate

Per-Site Breakdown

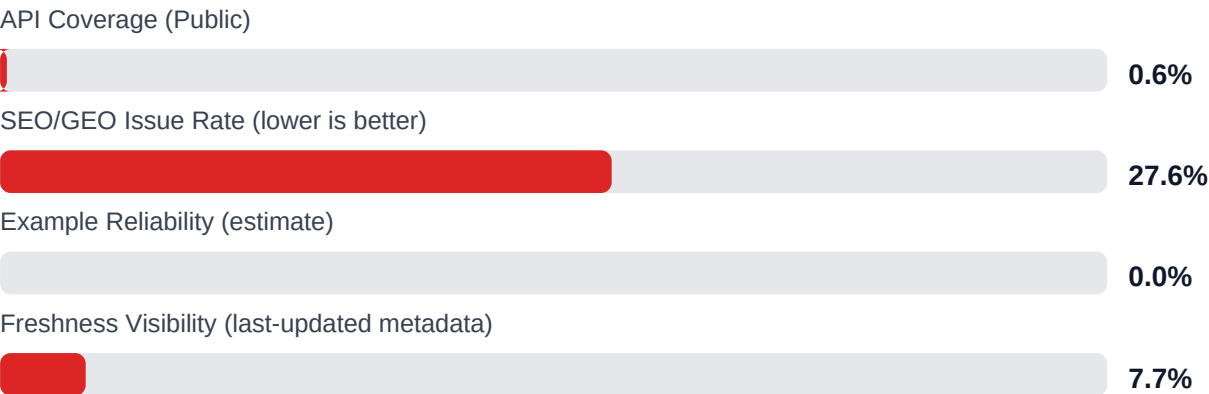
Site URL	Pages	Broken Links	API Cov %	Example Rel %	SEO Issue %
https://developers.fireblocks.com/	724	21	0.6	0.0	27.6

Industry Benchmark Comparison

Benchmark	Score	Gap
Top performers (Stripe, Twilio)	92+	-55
Industry average (developer docs)	85	-48
Minimum acceptable	70	-33
Your score	37	-

Gap to industry average: **48 points**. Top-performing developer documentation platforms (Stripe, Twilio, Vercel) score 92+. Closing this gap reduces support ticket volume by an estimated 20-30%.

Board-Level Metrics

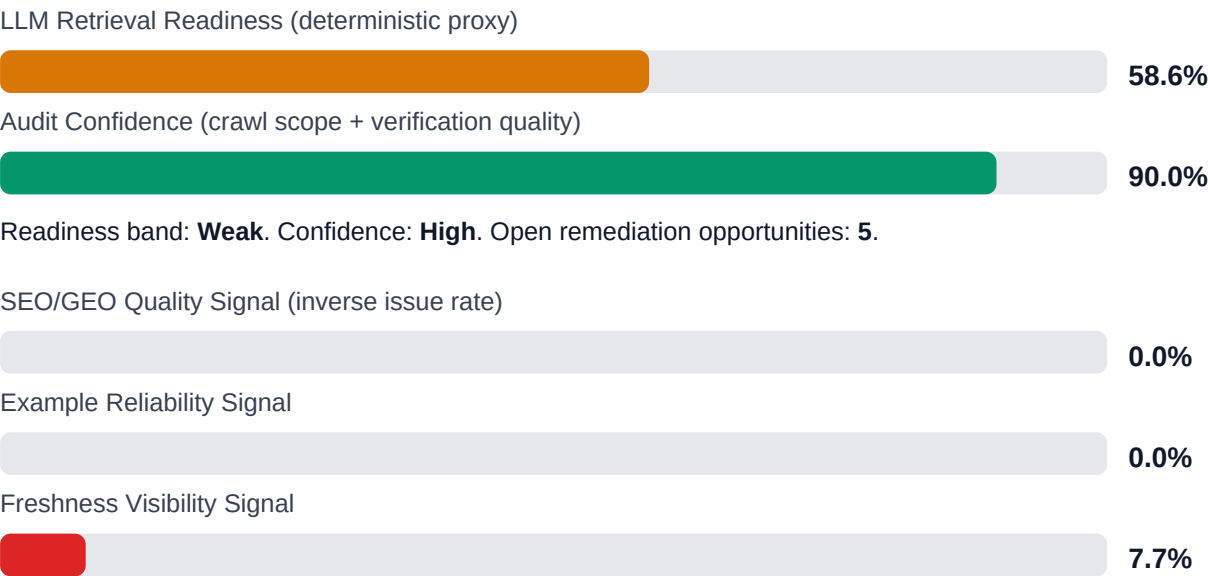


Internal Metrics (from scorecard)

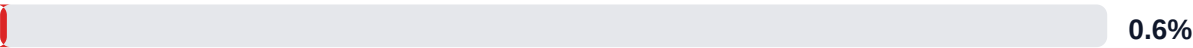


Pipeline Opportunity Fit (Deterministic)

This section maps cold-audit signals to problem classes that the pipeline can remediate directly. No LLM inference is used in this mapping.

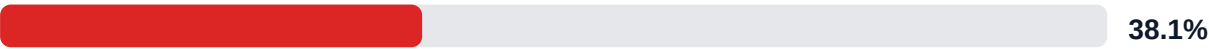


API Coverage Signal



LLM retrieval readiness breakdown (deterministic)

Chunkability



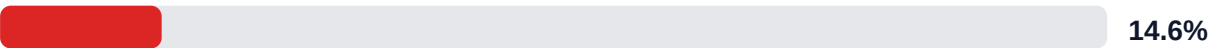
Answerability



Citationability



Stale Risk (inverse)



Evidence coverage: 73.7% (724/983 key claims backed by deterministic evidence signals).

Highest-value remediation targets

Problem	Severity	Pipeline solution path
Internal documentation link integrity issues	HIGH	Weekly link governance via automated audits and consolidated remediation queue
Search and AI readability quality gaps	HIGH	Deterministic SEO/GEO linting and fix cycles before publish
Code example reliability risk	HIGH	Snippet lint + smoke checks + protocol self-verify coverage
Weak freshness and lifecycle visibility	MEDIUM	Lifecycle management, stale detection, and weekly governance cadence
API reference coverage and drift risk	HIGH	API-first contract flow, validators, regression gates, and drift checks

Financial Exposure Model

Low Estimate \$386,267 annual exposure Remediation: \$1,805 Monthly loss: \$3,562 Opportunity: \$28,476/mo	Base Estimate \$480,137 annual exposure Remediation: \$4,702 Monthly loss: \$11,144 Opportunity: \$28,476/mo	High Estimate \$637,504 annual exposure Remediation: \$7,600 Monthly loss: \$24,016 Opportunity: \$28,476/mo
--	--	--

Remediation: finding-level effort model | Monthly loss: operational friction model | Opportunity: engineering/support/release delay

Risk Matrix

ID	Issue	Severity	Monthly Loss	Fix Cost
F-API-COVERAGE	Undocumented API operations	HIGH	\$2,508	\$950
F-EXAMPLES-RELIABILITY	Code examples fail or are non-runnable	HIGH	\$1,938	\$808
F-FRESHNESS	Documentation freshness debt	HIGH	\$969	\$475
F-DRIFT	Code/docs contract drift	MEDIUM	\$1,642	\$570
F-RETRIEVAL	RAG retrieval quality risk	MEDIUM	\$1,949	\$855
F-LAYERS	Missing required doc layers	LOW	\$1,539	\$712
F-TERMINOLOGY	Terminology inconsistency	LOW	\$598	\$332

Priority Action Plan (Next 14 Days)

ID	Finding	Severity
F-API-COVERAGE	Undocumented API operations	HIGH
F-EXAMPLES-RELIABILITY	Code examples fail or are non-runnable	HIGH
F-FRESHNESS	Documentation freshness debt	HIGH
F-DRIFT	Code/docs contract drift	MEDIUM
F-RETRIEVAL	RAG retrieval quality risk	MEDIUM

Recommended Actions

1. Add usage examples, request/response samples, or integration snippets to the 622 detected API pages to lift 0.61% reference coverage above 40% [impact: critical, effort: high]

2. Instrument last-updated timestamps on all pages to raise 7.73% freshness visibility above 80% and reduce 85.41% stale risk perception [impact: high, effort: low]
3. Fix all 21 confirmed broken internal links to restore navigation flow and recover SEO link equity [impact: high, effort: low]
4. Audit and fix detection for example code blocks (0% reliability on 724 pages) to enable validation and improve AI training corpus quality [impact: high, effort: medium]
5. Resolve 27.62% SEO/geo issues (missing meta descriptions, title tags, or structured data) to improve organic developer acquisition [impact: medium, effort: medium]

Expert Analysis: Strengths and Risks

Strengths

- + Complete crawl coverage at 100.14% with 724 pages indexed and no repository navigation link breakage
- + Perfect actionability coverage at 100% and citationability at 100% for AI retrieval
- + High overall audit confidence at 86.5% with strong crawl confidence at 95%
- + Large API surface detected (622 pages) provides foundation for coverage improvement
- + Majority of claims (73.65%) include supporting evidence across 983 detected assertions

Risks

- API usage documentation at 0.61% coverage creates massive onboarding friction and support burden for 622 API endpoints
- Zero detectable example reliability across 724 pages blocks developer self-service and AI answer quality
- 85.41% stale risk with only 7.73% freshness visibility erodes trust and may surface outdated integration patterns
- 21 broken internal links disrupt navigation and damage SEO authority across user-facing documentation
- 27.62% SEO/geo issue rate limits discoverability and organic acquisition of developer users
- 38.12% chunkability score constrains RAG and LLM citation quality for support automation
- 26.35% of key claims lack evidence (260 of 983), weakening credibility for compliance and enterprise buyers

Limitations

- * External audit cannot verify accuracy of technical claims, code correctness, or whether 622 API endpoints reflect current production behavior.
- * 0% example reliability likely reflects JavaScript rendering or non-standard code block markup; manual inspection required to confirm true example presence and quality.
- * 3 unverified links may be functional but blocked by authentication, rate limiting, or server configuration; manual testing needed for definitive status.
- * Audit cannot measure actual developer sentiment, support ticket attribution, or conversion impact of the 0.61% API coverage gap without internal analytics correlation.

Methodology

This audit applies a rigorous, repeatable methodology combining automated analysis with expert synthesis. The platform evaluates documentation quality across five pillars.

1. Automated Crawl + Structural HTML Analysis

Every public documentation page is crawled and analyzed for structural integrity, link health, navigation depth, and content organization. The crawler detects broken links, orphaned pages, redirect chains, and missing metadata.

2. SEO/GEO Scoring Model (24 Checks)

Each page is evaluated against 8 GEO checks (LLM/AI search optimization) and 16 SEO checks (traditional search engine optimization). Checks cover meta descriptions, heading hierarchy, fact density, URL structure, internal linking, and content depth.

3. API Reference Cross-Coverage Analysis

The platform compares published API reference documentation against the actual API surface (OpenAPI specs, GraphQL schemas, gRPC protos). Coverage gaps, undocumented endpoints, and stale references are identified.

4. Code Example Reliability Estimation

Code examples embedded in documentation are analyzed for syntactic correctness, completeness, and consistency with current API signatures. Reliability scores estimate the likelihood of examples working as documented.

5. Executive Synthesis and Prioritization

All quantitative findings are synthesized into actionable executive narratives. The analysis identifies patterns, prioritizes risks, and generates strategic recommendations calibrated to business impact.

Appendix A -- Economic Model Assumptions

Assumption	Value
engineer_hourly_usd	95.0
support_hourly_usd	65.0
release_count_per_month	8.0
baseline_manual_sync_hours_per_week	6.0
avg_release_delay_hours	18.0
monthly_support_tickets	450.0
docs_related_ticket_share	0.28
avg_ticket_handling_minutes	35.0
monthly_doc_influenced_evals	85.0
eval_to_customer_rate	0.18
avg_customer_monthly_value_usd	4200.0
docs_friction_revenue_sensitivity	0.22

Note: estimates are directional and should be calibrated with client rates, release cadence, and support volume.

Appendix B -- Evidence Traceability

Finding	Evidence Source	Confidence
F-API-COVERAGE -- Undocumented API operations	OpenAPI + docs text scan	HIGH
F-EXAMPLES-RELIABILITY -- Code examples fail or are non-runnable	examples_smoke_report.json	HIGH
F-FRESHNESS -- Documentation freshness debt	frontmatter date scan	HIGH
F-DRIFT -- Code/docs contract drift	pr_docs_contract.json + api_sdk_drift_report.json	HIGH
F-RETRIEVAL -- RAG retrieval quality risk	retrieval_evals_report.json	HIGH
F-LAYERS -- Missing required doc layers	policy pack + frontmatter layer scan	HIGH

F-TERMINOLOGY -- Terminology inconsistency	glossary.yml forbidden terms scan	LOW
--	-----------------------------------	-----

All findings are traceable to automated checks. Evidence sources reference specific crawl data, API specs, or code analysis results.